Data Structures and Algorithms -3 (25CS2103E)

Intelligent Job Portal

Review 2 Team 9, Section 6

Team Members:
2520030353 - Abhay Tripathi
2520030477 - K. Pranav

Under the Esteemed Guidance of
Dr. V. Sireesha
Academic Year 2026-27

Contents

- Abstract
- Introduction
- Literature Survey
- Problem Statement and Objectives
- Flow Chart
- Results
- Hardware and Software Requirements
- Conclusion
- Future Scope

Abstract

The Intelligent Job Portal is a Java application that helps job seekers discover suitable opportunities using Data Structures and Algorithms. The NovaHire browser interface connects to a local Java HTTP server and a catalogue of job records.

KMP, Z and Rabin-Karp support exact keyword search, while edit distance handles spelling variations. Recommendations compare a candidate's skills, preferred location and desired role to produce an explainable match score.

Skill planning evaluates useful additions to a candidate's profile. Graph algorithms allocate candidates to available jobs, and knapsack optimization selects opportunities within a time budget. The project demonstrates how DSA methods support recruitment search and decision-making.

Introduction

Job seekers often review many vacancies before finding a role that matches their skills and preferences. Basic keyword search can miss relevant opportunities when a query contains a spelling error or when several jobs share similar titles.

The Intelligent Job Portal organizes job titles, companies, locations, required skills and salary information in one catalogue. Users can search vacancies, compare recommendations and identify skills that improve their match.

The system combines a browser interface with custom Java data structures and algorithm services. Recruiter and administrator views support the demonstration. The current version runs locally with CSV seed data and browser storage.

Literature Survey

Exact string matching: KMP and Z use prefix information to avoid repeated comparisons. Rabin-Karp compares rolling hashes and verifies matching substrings. These methods support keyword discovery in job records.

Approximate matching: Wagner-Fischer edit distance measures character changes between strings. It helps recover job titles from misspelled queries.

Ranking and planning: Weighted attribute scores explain recommendation order. Knapsack dynamic programming selects a valuable subset within a fixed budget.

Allocation: Edmonds-Karp computes maximum flow in a candidate-job network. These established DSA methods provide the algorithmic foundation for the portal.

Problem Statement and Objectives

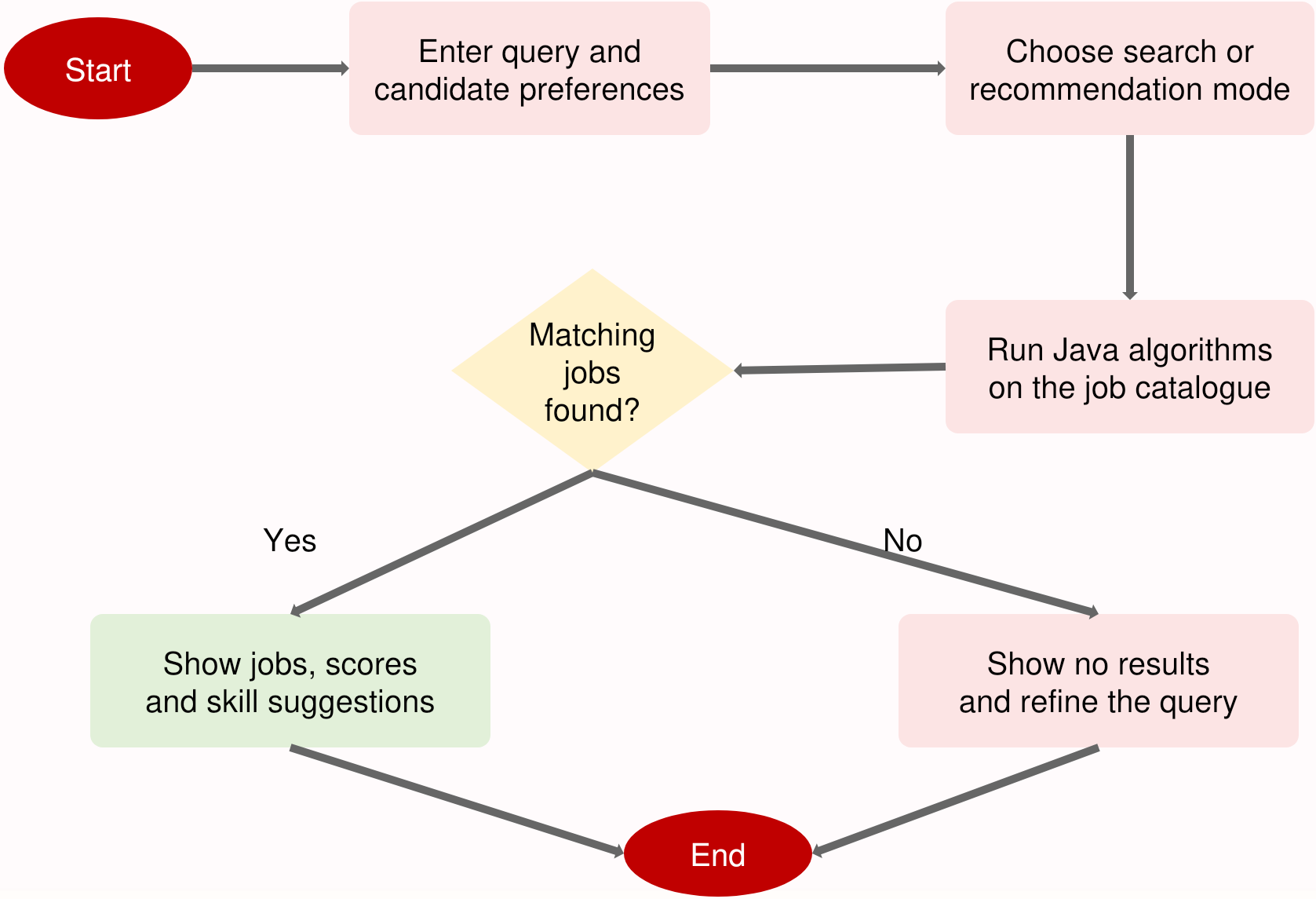
Problem Statement

Manual vacancy screening takes time, and exact keywords alone do not explain how well a job fits a candidate. A structured portal is needed to search records and compare opportunities using skills and preferences.

Objectives

- Organize job records in a searchable catalogue.
- Support exact search and recovery from spelling errors.
- Rank jobs using skills, location and desired role.
- Suggest skill additions and plan applications within a budget.
- Demonstrate candidate allocation using graph algorithms.
- Verify algorithm correctness through console and API tests.

Flow Chart


Start
Enter query and candidate preferences
Choose search or recommendation mode
Run Java algorithms on the job catalogue
Matching jobs found?
Yes
No
Show jobs, scores and skill suggestions
Show no results and refine the query
End

Results

The verified demonstration uses 30 seed job records. All 32 built-in self-tests and 15 HTTP integration checks passed.

Exact search: KMP, Z and Rabin-Karp each returned 14 matching jobs for “Java”. A misspelled “Pythn Developer” query also recovered relevant results.

Recommendations: A Java, Spring, SQL and AWS profile with Hyderabad as the preferred location gave Job 1 a 100/100 match.

Planning: Adding Hibernate and REST increased the tested aggregate recommendation score from 585 to 705. An 8-hour knapsack example selected four jobs with total value 296.

These results describe a local demonstration. Jobs published through the recruiter API remain in memory, and full browser end-to-end testing remains future work.

Hardware and Software Requirements

Hardware Requirements

Suggested setup: Intel Core i3 or equivalent processor
RAM: 4 GB or more
Storage: Space for the JDK, source code and project files
Desktop or laptop with keyboard, mouse and display
Internet: For initial setup and repository access

Software Requirements

Operating system: Windows, Linux or macOS
Backend: Java JDK 17 with the JDK HTTP server
Frontend: HTML, CSS and JavaScript in a modern browser
Storage: CSV job catalogue and browser localStorage
IDE: VS Code, IntelliJ IDEA or Eclipse
Version control: Git and GitHub

Conclusion

The Intelligent Job Portal applies DSA concepts to a working recruitment demonstration. Search algorithms retrieve job records, edit distance handles spelling variations, and weighted scoring explains recommendation order.

Dynamic programming supports skill planning and budgeted selection. Maximum flow demonstrates candidate allocation under job capacity constraints. The Java backend connects these routines to the NovaHire browser interface.

The verified tests confirm expected algorithm and API behavior on the seed catalogue. The project provides a useful foundation for a larger portal, with authentication, durable storage and broader user testing planned for future development.

Future Scope

- Add secure accounts for job seekers, recruiters and administrators.
- Store jobs, profiles and applications in a persistent database.
- Import resumes with user review of extracted skills.
- Expand skill matching with synonyms and role-specific criteria.
- Add real application tracking, job alerts and recruiter analytics.
- Improve indexing and test performance on larger catalogues.
- Complete browser, accessibility and mobile usability testing.

Thank you

Intelligent Job Portal
Team 9